

# Scallop Token

## Audit Report

---

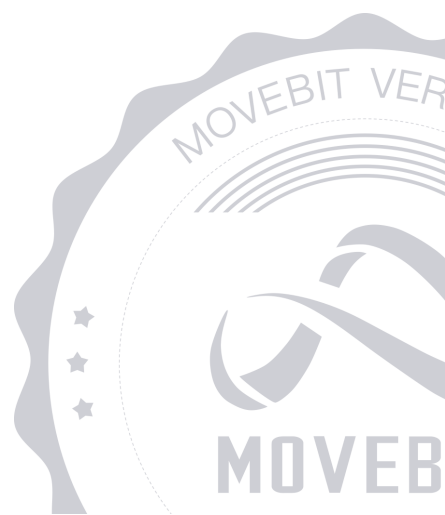


[contact@movebit.xyz](mailto:contact@movebit.xyz)



[https://twitter.com/movebit\\_](https://twitter.com/movebit_)

Tue Jan 23 2024



# Scallop Token Audit Report

---

## 1 Executive Summary

### 1.1 Project Information

|             |                                                                                                       |
|-------------|-------------------------------------------------------------------------------------------------------|
| Description | The native cryptographically-secure fungible protocol token of Scallop                                |
| Type        | Token                                                                                                 |
| Auditors    | MoveBit                                                                                               |
| Timeline    | Mon Jan 15 2024 – Tue Jan 23 2024                                                                     |
| Languages   | Move                                                                                                  |
| Platform    | Sui                                                                                                   |
| Methods     | Architecture Review, Unit Testing, Manual Review                                                      |
| Source Code | <a href="https://github.com/scallop-io/scallop-token">https://github.com/scallop-io/scallop-token</a> |
| Commits     | <a href="#">433dca897a26ee6f325d83b21b21517d552372d754b233117fad80b56da8c918a08f4420a91f130b</a>      |

## 1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

| ID    | File                                            | SHA-1 Hash                               |
|-------|-------------------------------------------------|------------------------------------------|
| XST   | x_supply/tests/x_supply_test.move               | b199296c92be9b8b87efa443cb2b6ef0aee60275 |
| MOV   | x_supply/Move.toml                              | 90c67fdb98dec42ceac73bd00c19f5d883ce2dfe |
| XSU   | x_supply/sources/x_supply.move                  | 03b811b8d2f5790b9a69445cee8229391aeb921e |
| BVS   | linear_ve_sca/tests/burn_ve_sca.move            | 079f8013a6884effa4ad14a38c026b3a5c0773c2 |
| CFT   | linear_ve_sca/tests/constants_for_test.move     | dbc1a8605f63a0e6740739bbd1cb0faf647fb697 |
| PRVS  | linear_ve_sca/tests/partial_redeem_ve_sca.move  | e22b8f560f6607cfa1929046473e665871cfc050 |
| SET   | linear_ve_sca/tests/setup.move                  | d59a3ac5c66ee13364dc94c7364719a1739d488e |
| MVS   | linear_ve_sca/tests/mint_ve_sca.move            | 991f125a5a3651b44d7185dc4d69fc1e0e64c232 |
| OTRVS | linear_ve_sca/tests/one_time_redeem_ve_sca.move | 356161db2b859eeca2af3e060ba2da10523db99f |
| MOV1  | linear_ve_sca/Move.toml                         | 2eca47fa6d5208cf0134996fa320e965cc70e5c3 |
| DIS   | linear_ve_sca/sources/display.move              | 5d35622be9a4a078bb567dddd6a0bc2df98f9cb1 |
| VMO   | linear_ve_sca/sources/vesting_model.move        | dc451e7153a76228456cdc7f60daad5ff075a114 |

|      |                                          |                                          |
|------|------------------------------------------|------------------------------------------|
| LVS  | linear_ve_sca/sources/linear_ve_sca.move | b4ff674a932ffee82764693e4799635850c831d0 |
| TID  | linear_ve_sca/sources/typed_id.move      | b45fd2076a8015d9047a5f3868cfca261efee9ea |
| VER  | linear_ve_sca/sources/version.move       | ff1090283c75a20a65f2ad55928f5684d6e3789c |
| MOV2 | sca/Move.toml                            | 98fa358ddd5ad471d8dc96796a9eb037c55b650c |
| SCA  | sca/sources/sca.move                     | b82fa6fdb0170b6061989648af2fbd3119cecd01 |

### 1.3 Issue Statistic

| Item          | Count | Fixed | Acknowledged |
|---------------|-------|-------|--------------|
| Total         | 7     | 7     | 0            |
| Informational | 1     | 1     | 0            |
| Minor         | 2     | 2     | 0            |
| Medium        | 1     | 1     | 0            |
| Major         | 3     | 3     | 0            |
| Critical      | 0     | 0     | 0            |

## 1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security–related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction–ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

## 1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

### (1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

### (2) Code Review

The code scope is illustrated in section 1.2.

### (3) Formal Verification

Perform formal verification for key functions with the Move Prover.

### (4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

## 2 Summary

This report has been commissioned by [Scallop Token](#) to identify any potential issues and vulnerabilities in the source code of the [Scallop Token](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 7 issues of varying severity, listed below.

| ID    | Title                                                                                                                                                   | Severity      | Status |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|--------|
| LVS-1 | Lack of Contract Version Check in Key Functions of <code>linear_ve_sca</code> Module                                                                    | Major         | Fixed  |
| LVS-2 | <code>update_vesting_model</code> Should Consider the Already Minted <code>VeSca</code>                                                                 | Major         | Fixed  |
| LVS-3 | Insufficient Balance Error in <code>redeem_sca()</code> after Full Withdrawal in <code>withdraw_sca()</code> Function                                   | Medium        | Fixed  |
| SCA-1 | Secure Handling of CoinMetadata in <code>SCA</code> Token Creation                                                                                      | Major         | Fixed  |
| VER-1 | Optimizing Version Update Handling in the <code>upgrade_version()</code> Function                                                                       | Informational | Fixed  |
| VMO-1 | Missing <code>vesting_period</code> Validation in <code>add_model()</code> and <code>update_model()</code> Functions in <code>vesting_model.move</code> | Minor         | Fixed  |
| XSU-1 | Identification of Unused Constants                                                                                                                      | Minor         | Fixed  |



|  |                                           |  |  |
|--|-------------------------------------------|--|--|
|  | and Modules in <code>x_supply.move</code> |  |  |
|--|-------------------------------------------|--|--|

## 3 Participant Process

Here are the relevant actors with their respective abilities within the [Scallop Token](#) Smart Contract:

### Admin

- Admin can add a vesting model through `add_vesting_model()`.
- Admin can withdraw `SCA` tokens from the treasury through `withdraw_sca()`.
- Admin can add an issuer through `add_issuer()`.
- Admin can remove an issuer through `remove_issuer()`.
- Admin can update the contract version through `update_version()`.

### User

- User can mint `LinearVeSca` tokens through `mint()`.
- User can redeem vested `SCA` tokens through `redeem_sca()`.
- User can burn the `LinearVeSca` token through `burn()`.
- User can deposit `SCA` tokens into the treasury through `deposit_sca()`.

## 4 Findings

### LVS-1 Lack of Contract Version Check in Key Functions of `linear_ve_sca` Module

Severity: Major

Status: Fixed

Code Location:

`linear_ve_sca/sources/linear_ve_sca.move#270,286,305,312,324,334`

Descriptions:

Some critical functions in the `linear_ve_sca` module lack a contract version check, which may lead to the execution of unsafe code. These functions include

`add_vesting_model()`, `update_vesting_model()`, `deposit_sca()`, `withdraw_sca()`, `add_issuer()`, and `remove_issuer()`.

Suggestion:

It is recommended to add a contract version check.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

## LVS-2 `update_vesting_model` Should Consider the Already Minted `VeSca`

Severity: Major

Status: Fixed

Code Location:

`linear_ve_sca/sources/linear_ve_sca.move#286-302`

Descriptions:

The `update_vesting_model` function updates the `cliff` and `vesting_period` values of the `VestingModel`. However, when updating the `VestingModel`, it doesn't consider the already minted `VeSca`. Suppose `enable_partial_redeem` is set to `true`, then users might call `redeem_sca` after the `VestingModel` update and receive fewer `SCA` than before, potentially leading to overflow and calculation issues.

```
let potential_redeemable_amount = potential_vested_amount -  
ve_sca.redeemed_amount;
```

Additionally, if `enable_partial_redeem` is set to `false`, users might inadvertently redeem `SCA` based on the previous `VestingModel`, possibly resulting in a loss of `SCA` (as per the previous `VestingModel`, they might have reached the time to redeem all `SCA`, but not under the updated `VestingModel`).

```
let deduct_balance = if (ve_sca.enable_partial_redeem) {  
  // When the ve_sca allows partial redeem, only deduct the redeemable amount  
  balance::split(&mut ve_sca.balance, potential_redeemable_amount)  
} else {  
  // When the ve_sca does not allow partial redeem, deduct all the balance  
  let all_balance_value = balance::value(&ve_sca.balance);  
  balance::split(&mut ve_sca.balance, all_balance_value)  
};
```

Suggestion:

It is recommended to consider the already minted `VeSca` when updating the `VestingModel`, especially in terms of computational issues, overflow problems, and potential financial losses for users.

### Resolution:

This issue has been fixed. The client has already removed the function `update_vesting_model`.

## LVS-3 Insufficient Balance Error in `redeem_sca()` after Full Withdrawal in `withdraw_sca()` Function

Severity: Medium

Status: Fixed

Code Location:

`linear_ve_sca/sources/linear_ve_sca.move#312`

Descriptions:

In the function `withdraw_sca()`, if all amounts are withdrawn, using the `redeem_sca()` function again to claim tokens will result in an insufficient balance error. Users can only proceed to claim tokens once the administrator recharges the balance to a sufficient level.

Suggestion:

It is recommended to verify that the balance is sufficient to pay for user withdrawals.

Resolution:

This issue has been fixed. The client has added verification when claiming their tokens.

# SCA-1 Secure Handling of CoinMetadata in SCA Token Creation

Severity: Major

Status: Fixed

Code Location:

sca/sources/sca.move#91

Descriptions:

After creating an SCA Token, the return value CoinMetadata is obtained, representing certain information about the currency. In the code, this information is shared, which is insecure. Unless there is an intention to modify currency information, it is recommended to freeze it using public\_share\_object().

```
fun init(otw: SCA, ctx: &mut TxContext) {  
    ...  
    transfer::public_share_object(coinMeta);  
}
```

The following is the official repository code for the SUI token:

```
fun new(ctx: &mut TxContext): Balance<SUI> {  
    .....  
    let (treasury, metadata) = coin::create_currency(  
        SUI {},  
        9,  
        b"SUI",  
        b"Sui",  
        // TODO: add appropriate description and logo url  
        b"",  
        option::none(),  
        ctx  
    );  
    transfer::public_freeze_object(metadata);  
    .....  
}
```

Suggestion:

It is recommended to freeze coinMeta using public\_share\_object().

### Resolution:

This issue has been fixed. The client has adopted our suggestions.



# VER-1 Optimizing Version Update Handling in the `upgrade_version()` Function

**Severity:** Informational

**Status:** Fixed

**Code Location:**

`linear_ve_sca/sources/version.move#25`

**Descriptions:**

In the function `upgrade_version()`, the method of incrementing the version number by 1 each time is used for version updates. However, if the value of `CURRENT_VERSION` in the upgraded code contract is significantly higher than the previous version, for example, if the previous version is `5` and the code is updated with `CURRENT_VERSION` changed to `10`, calling `upgrade_version()` again after the contract upgrade will only increase the version by `1`. The version check will fail, requiring multiple consecutive calls (in this case, 5) to achieve version consistency. This may not be an optimal approach. We recommend directly setting `version.value` to `CURRENT_VERSION` in the `upgrade_version()` function and asserting that `version.value < CURRENT_VERSION` before modification to ensure that the new version is greater than the old version. This approach may be more effective.

**Suggestion:**

It is recommended to set `version.value` directly to `CURRENT_VERSION`.

**Resolution:**

This issue has been fixed. The client has adopted our suggestions.

## VMO-1 Missing `vesting_period` Validation in `add_model()` and `update_model()` Functions in `vesting_model.move`

Severity: Minor

Status: Fixed

Code Location:

`linear_ve_sca/sources/vesting_model.move#35,51`

Descriptions:

In `vesting_model.move`, the functions `add_model()` and `update_model()` lack validation for the `vesting_period`. Since `vesting_period` is used as a divisor in calculating the extractable amount later on, it should be validated during its setting to prevent it from being equal to 0.

Suggestion:

It is recommended to add an assertion to validate that `vesting_period` is not equal to 0.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

# XSU-1 Identification of Unused Constants and Modules in `x_supply.move`

Severity: Minor

Status: Fixed

Code Location:

`x_supply/sources/x_supply.move#10,13`

Descriptions:

In `x_supply.move`, the constant `ERequire0TW` is not being used, and similarly, the imported module `sui::types` is also unused. This could result in unnecessary gas consumption and impact code readability.

Suggestion:

It is recommended to remove redundant code.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

# Appendix 1

## Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non–exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

## Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

# Appendix 2

## Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

